

此前各章已结合具体的数据结构，循序渐进地介绍过多种基本的排序算法：2.8节和3.5节分别针对向量和列表，统一以排序器的形式实现过起泡排序、归并排序、插入排序以及选择排序等算法；9.4.1节也曾按照散列的思路与手法，实现过桶排序算法；9.4.3节还将桶排序推广至基数排序算法；10.2.5节也曾完美地利用完全二叉堆的特长，实现过就地堆排序算法。

本章着重于高级排序算法。与以上基本算法一样，其构思与技巧各具特色，在不同应用中的效率也各有千秋。因此在学习过程中，唯有更多地关注不同算法之间细微而本质的差异，留意体会其优势与不足，方能做到运用自如，并结合实际问题的需要，合理取舍与并适当改造。

§ 12.1 快速排序

12.1.1 分治策略

与归并排序算法一样，快速排序（quicksort）算法^①也是分治策略的典型应用，但二者之间也有本质区别。2.8.3节曾指出，归并排序的计算量主要消耗于有序子向量的归并操作，而子向量的划分却几乎不费时间。快速排序恰好相反，它可以在 $O(1)$ 时间内，由子问题的解直接得到原问题的解；但为了将原问题划分为两个子问题，却需要 $O(n)$ 时间。

快速排序算法虽然能够确保，划分出来的子任务彼此独立，并且其规模总和保持渐进不变，却不能保证两个子任务的规模大体相当——实际上，甚至有可能极不平衡。因此，该算法并不能保证最坏情况下的 $O(n \log n)$ 时间复杂度。尽管如此，它仍然受到人们的青睐，并在实际应用中往往成为首选的排序算法。究其原因在于，快速排序算法易于实现，代码结构紧凑简练，而且对于按通常规律随机分布的输入序列，快速排序算法实际的平均运行时间较之同类算法更少。

下面结合向量介绍该算法的原理，并针对实际需求相应地给出不同的实现版本。

12.1.2 轴点

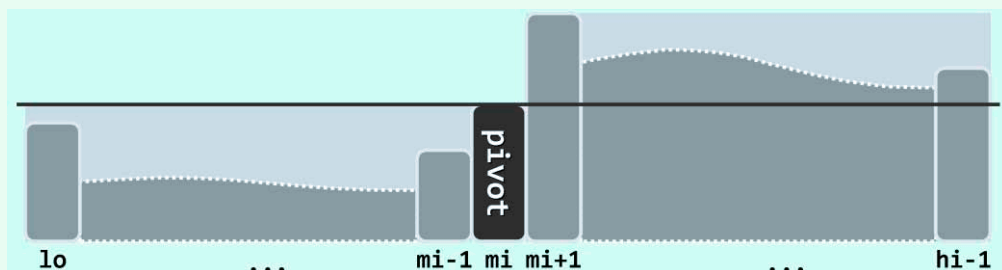


图12.1 序列的轴点（这里用高度来表示各元素的大小）

^① 由英国计算机科学家、1980年图灵奖得主C. A. R. Hoare爵士于1960年发明^[64]

如图12.1所示，考查任一向量区间 $S[lo, hi)$ 。对于任何 $lo \leq mi < hi$ ，以元素 $S[mi]$ 为界，都可分割出前、后两个子向量 $S[lo, mi)$ 和 $S(mi, hi)$ 。若 $S[lo, mi)$ 中的元素均不大于 $S[mi]$ ，且 $S(mi, hi)$ 中的元素均不小于 $S[mi]$ ，则元素 $S[mi]$ 称作向量 S 的一个轴点（pivot）。

设向量 S 经排序可转化为有序向量 S' 。不难看出，轴点位置 mi 必然满足如下充要条件：

- a) $S[mi] = S'[mi]$
- b) $S[lo, mi)$ 和 $S'[lo, mi)$ 的成员完全相同
- c) $S(mi, hi)$ 和 $S'(mi, hi)$ 的成员完全相同

因此，不仅以轴点 $S[mi]$ 为界，前、后子向量的排序可各自独立地进行，而且更重要的是，一旦前、后子向量各自完成排序，即可立即（在 $O(1)$ 时间内）得到整个向量的排序结果。

采用分治策略，递归地利用轴点的以上特性，便可完成原向量的整体排序。

12.1.3 快速排序算法

按照以上思路，可作为向量的一种排序器，实现快速排序算法如代码12.1所示。

```
1 template <typename T> //向量快速排序
2 void Vector<T>::quickSort(Rank lo, Rank hi) { //0 <= lo < hi <= size
3     if (hi - lo < 2) return; //单元素区间自然有序，否则...
4     Rank mi = partition(lo, hi - 1); //在[lo, hi - 1]内构造轴点
5     quickSort(lo, mi); //对前缀递归排序
6     quickSort(mi + 1, hi); //对后缀递归排序
7 }
```

代码12.1 向量的快速排序

可见，轴点的位置一旦确定，则只需以轴点为界，分别递归地对前、后子向量实施快速排序；子向量的排序结果就地返回之后，原向量的整体排序即告完成。算法的核心与关键在于：

轴点构造算法`partition()`应如何实现？可以达到多高的效率？

12.1.4 快速划分算法

■ 反例

事情远非如此简单，我们首先遇到的困难就是，并非每个向量都必然含有轴点。以如图12.2所示长度为9的向量为例，不难验证，其中任何元素都不是轴点。

1	2	3	4	5	6	7	8	0
0	1	2	3	4	5	6	7	8

图12.2 有序向量经循环左移一个单元后，将不含任何轴点

事实上根据此前的分析，任一元素作为轴点的必要条件之一是，其在初始向量 S 与排序后有序向量 S' 中的秩应当相同。因此反过来一般地，只要向量中所有元素都是错位的——即所谓的错排序列——则任何元素都不可能是轴点。

由上可见，若保持原向量的次序不变，则不能保证总是能够找到轴点。因此反过来，唯有通过适当地调整向量中各元素的位置，方可“人为地”构造出一个轴点。

■ 思路

为在区间 $[lo, hi]$ 内构造出一个轴点，首先需要任取某一元素 m 作为“培养对象”。

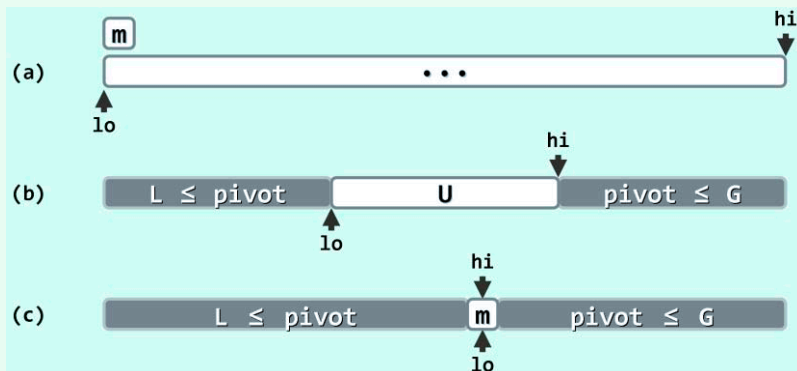


图12.3 轴点构造算法的构思

如图12.3(a)所示，不妨取首元素 $m = S[lo]$ 作为候选，将其从向量中取出并做备份，腾出的空闲单元便于其它元素的位置调整。然后如图(b)所示，不断试图移动 lo 和 hi ，使之相互靠拢。当然，整个移动过程中，需始终保证 lo (hi) 左侧 (右侧) 的元素均不大于 (不小于) m 。

最后如图(c)所示，当 lo 与 hi 彼此重合时，只需将原备份的 m 回填至这一位置，则 $S[lo = hi] = m$ 便成为一个名副其实的轴点。

以上过程在构造出轴点的同时，也按照相对于轴点的大小关系，将原向量划分为左、右两个子向量，故亦称作快速划分 (quick partitioning) 算法。

■ 实现

按照以上思路，快速划分算法可实现如代码12.2所示。

```
1 template <typename T> //轴点构造算法：通过调整元素位置构造区间 $[lo, hi]$ 的轴点，并返回其秩
2 Rank Vector<T>::partition(Rank lo, Rank hi) { //版本A：基本形式
3     swap(_elem[lo], _elem[lo + rand() % (hi - lo + 1)]); //任选一个元素与首元素交换
4     T pivot = _elem[lo]; //以首元素为候选轴点——经以上交换，等效于随机选取
5     while (lo < hi) { //从向量的两端交替地向中间扫描
6         while ((lo < hi) && (pivot <= _elem[hi])) //在不小于pivot的前提下
7             hi--; //向左拓展右端子向量
8         _elem[lo] = _elem[hi]; //小于pivot者归入左侧子序列
9         while ((lo < hi) && (_elem[lo] <= pivot)) //在不大于pivot的前提下
10            lo++; //向右拓展左端子向量
11        _elem[hi] = _elem[lo]; //大于pivot者归入右侧子序列
12    } //assert: lo == hi
13    _elem[lo] = pivot; //将备份的轴点记录置于前、后子向量之间
14    return lo; //返回轴点的秩
15 }
```

代码12.2 轴点构造算法 (版本A)

为便于和稍后的改进版本进行比较，不妨称作版本A。

过程

可见，算法的主体框架为循环迭代；主循环的内部，通过两轮迭代交替地移动`lo`和`hi`。

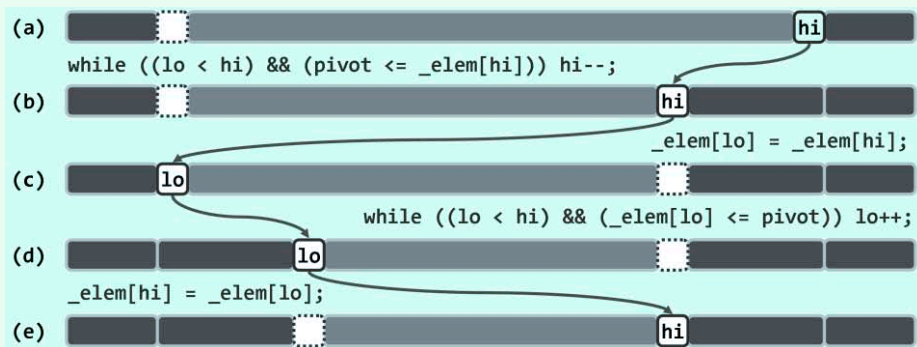


图12.4 轴点构造过程

各迭代的初始状态如图12.4(a)所示。反复地将候选轴点`pivot`与当前的`_elem[hi]`做比较，只要前者不大于后者，就不断向左移动`hi`（除非`hi`即将越过`lo`）。`hi`无法移动继续时，当如图(b)所示。于是接下来如图(c)所示，将`_elem[hi]`转移至`_elem[lo]`，并归入左侧子向量。

随后对称地，将`_elem[lo]`与`pivot`做比较，只要前者不大于后者，就不断向右移动`lo`（除非`lo`即将越过`hi`）。`lo`无法继续移动时，当如图(d)所示。于是接下来如图(e)所示，将`_elem[lo]`转移至`_elem[hi]`，并归入右侧子向量。

每经过这样的两轮移动，`lo`与`hi`的间距都会缩短，故该算法迟早会终止。当然，若如图(e)所示`lo`与`hi`仍未重合，则可再做两轮移动。不难验证，在任一时刻，在以`lo`和`hi`为界的三个子向量中，左、右子向量分别满足12.1.2节所列的轴点充要条件b)和c)。而随着算法的持续推进，中间子向量的范围则不断压缩。当主循环退出时`lo`和`hi`重合，充要条件a)也随即满足。至此，只需将`pivot`“镶嵌”于左、右子向量之间，即实现了对原向量的一次轴点划分。

该算法的运行时间线性正比于被移动元素的数目，线性正比于原向量的规模 $O(hi - lo)$ 。

实例

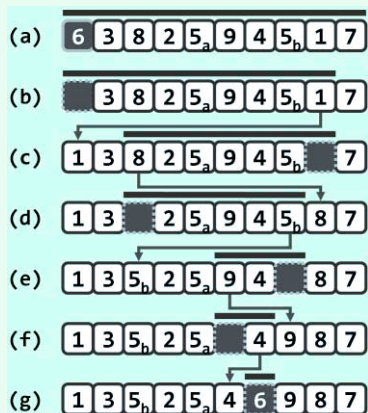


图12.5 轴点构造算法实例

快速划分算法的一次完整运行过程，如图12.5所示。输入序列A如图(a)长度为10，选择`A[0] = 6`作为轴点候选。以下，`hi`和`lo`的第一趟交替移动的过程及结果如图(b~c)所示，第二趟交替移动的过程及结果如图(d~e)所示，最后一趟交替移动的过程及结果如图(f~g)所示。

由于`lo`和`hi`的移动方向相反，故原处于向量右（左）端较小（大）的元素将按颠倒的次序转移至左（右）端；特别地，重复的元素也将按颠倒的次序转移至相对的一端，因而不保持其原有的相对次序。由此可见，如此实现的快速排序算法并不稳定。从图12.5实例中数值为5的两个元素的移动过程与最终效果，不难看出这一点。

12.1.5 复杂度

■ 最坏情况

上节的分析结论指出，采用代码12.2中的`partition()`算法，可在线性时间内将原向量的排序问题分解为两个相互独立、总体规模保持线性的子向量排序问题；而且根据轴点的性质，由各自排序后的子向量，可在常数时间内得到整个有序向量。也就是说，分治策略得以高效实现的两个必要条件——子问题划分的高效性及其相互之间的独立性——均可保证。然而尽管如此，另一项关键的必要条件——子任务规模接近——在这里却无法保证。事实上，由`partition()`算法划分出的子任务在规模上不仅不能保证接近，而且可能相差悬殊。

反观`partition()`算法不难发现，其划分所得子序列的长度与划分的具体过程无关，而是完全取决于入口处所选的候选轴点。具体地，若在最终有序向量中该候选元素的秩为 r ，则子向量的规模必为 r 和 $n - r - 1$ 。特别地， $r = 0$ 时子向量规模分别为 0 和 $n - 1$ ——左侧子向量为空，而右侧子向量与原向量几乎等长。当然，对称的 $r = n - 1$ 亦属最坏情况。

更糟糕的是，这类最坏情况可能持续发生。比如，若每次都是简单地选择最左端元素`_elem[lo]`作为候选轴点，则对于完全（或几乎完全）有序的输入向量，每次（或几乎每次）划分的结果都是如此。这种情况下，若将快速排序算法处理规模为 n 的向量所需的时间记作 $T(n)$ ，则如下递推关系始终成立：

$$T(n) = T(0) + T(n - 1) + O(n) = T(n - 1) + O(n)$$

综合考虑到其常数复杂度的递归基，与以上递推关系联立即可解得：

$$T(n) = T(n - 2) + 2 \cdot O(n) = \dots = T(0) + n \cdot O(n) = O(n^2)$$

也就是说，其效率居然低到与起泡排序相近。

■ 降低最坏情况概率

那么，如何才能降低上述最坏情况出现的概率呢？读者可能已注意到，代码12.2的`partition()`算法在入口处增加了`swap()`一句，在区间内任选一个元素与`_elem[lo]`交换。就其效果而言，这使得后续的处理等同于随机选择一个候选轴点，从而在一定程度上降低上述最坏情况出现的概率。这种方法称作随机法。

类似地，也可采用所谓三者取中法：从待排序向量中任取三个元素，将数值居中者作为候选轴点。理论分析及实验统计均表明，较之固定选取某个元素或随机选取单个元素的策略，如此选出的轴点在最终有序向量中秩过小或过大的概率更低——尽管还不能彻底杜绝最坏情况。

■ 平均运行时间

以上关于最坏情况下效率仅为 $O(n^2)$ 的结论不免令人沮丧，难道快速排序名不副实？实际上，更为细致的分析与实验统计都一致地显示，在大多数情况下，快速排序算法的平均效率依然可以达到 $O(n \log n)$ ；而且较之其它排序算法，其时间复杂度中的常系数更小。以下就以最常见的场景为例，对采用随机法确定候选轴点的快速排序算法的平均效率做一估算。

假设待排序的元素服从独立均匀随机分布。于是，`partition()`算法在经过 $n - 1$ 次比较和至多 $n + 1$ 次移动操作之后，对规模为 n 的向量的划分结果无非 n 种可能，划分所得左侧子序列的长度分别是 $0, 1, \dots, n - 1$ ，分别决定于所取候选元素在最终有序序列中的秩。按假定条件，每种情况的概率均为 $1/n$ ，故若将算法的平均运行时间记作 $\hat{T}(n)$ ，则有：

$$\begin{aligned}\hat{T}(n) &= (n+1) + (1/n) \times \sum_{k=1}^n [\hat{T}(k-1) + \hat{T}(n-k)] \\ &= (n+1) + (2/n) \times \sum_{k=1}^n \hat{T}(k-1)\end{aligned}$$

等式两侧同时乘以 n ，则有：

$$n \cdot \hat{T}(n) = (n+1) \cdot n + 2 \cdot \sum_{k=1}^n \hat{T}(k-1)$$

以及同理：

$$(n-1) \cdot \hat{T}(n-1) = (n-1) \cdot n + 2 \cdot \sum_{k=1}^{n-1} \hat{T}(k-1)$$

以上两式相减，即得：

$$\begin{aligned}n \cdot \hat{T}(n) - (n-1) \cdot \hat{T}(n-1) &= 2n + 2 \cdot \hat{T}(n-1) \\ n \cdot \hat{T}(n) &= (n+1) \cdot \hat{T}(n-1) + 2n \\ \hat{T}(n)/(n+1) &= \hat{T}(n-1)/n + 2/(n+1) \\ &= \hat{T}(n-2)/(n-1) + 2/(n+1) + 2/n \\ &= \hat{T}(n-3)/(n-2) + 2/(n+1) + 2/n + 2/(n-1) \\ &= \dots \\ &= \hat{T}(0)/1 + 2/(n+1) + 2/n + 2/(n-1) + \dots + 2/2 \\ &= 2 \cdot \sum_{k=1}^{n+1} (1/k) - 1 \\ &\stackrel{②}{=} O(2 \cdot \ln n) = O(2 \cdot \ln 2 \cdot \log_2 n) = O(1.386 \cdot \log_2 n)\end{aligned}$$

正因为其良好的平均性能，加上其形象直观和易于实现的特点，快速排序算法自诞生起就一直受到人们的青睐，并被集成到Linux和STL等环境中。

12.1.6 应对退化

■ 重复元素

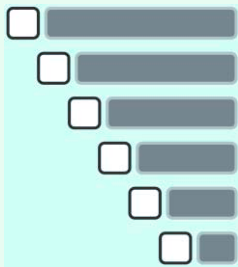


图12.6 partition()算法的退化情况，也是最坏情况

考查所有（或几乎所有）元素均重复的退化情况。对照代码12.2不难发现，partition()算法的版本A对此类输入的处理完全等效于此所举的最坏情况。事实上对于此类向量，主循环内部前一子循环的条件中“pivot <= _elem[hi]”形同虚设，故该子循环将持续执行，直至“lo < hi”不再满足。当然，在此之后另一内循环及主循环也将随即结束。

如图12.6所示，如此划分的结果必然是以最左端元素为轴点，原向量被分为极不对称的两个子向量。更糟糕的是，这一最坏情况还可能持续发生，从而使整个算法过程等效地退化为线性递归，递归深度为 $O(n)$ ，导致总体运行时间高达 $O(n^2)$ 。

② 若记 $h(n) = 1 + 1/2 + 1/3 + \dots + 1/n$ ，则有 $\ln(n+1) = \int_{i=1}^{n+1} (1/x) < h(n) < 1 + \int_{i=1}^n (1/x) = 1 + \ln n$

当然，可以在每次深入递归之前做统一核验，若确属退化情况，则无需继续递归而直接返回。但在重复元素不多时，如此不仅不能改进性能，反而会增加额外的计算量，总体权衡后得不偿失。

■ 改进

轴点构造算法可行的一种改进方案如代码12.3所示。为与如代码12.2所示同名算法版本A相区别，不妨称作版本B。

```

1 template <typename T> //轴点构造算法：通过调整元素位置构造区间[lo, hi]的轴点，并返回其秩
2 Rank Vector<T>::partition(Rank lo, Rank hi) { //版本B：可优化处理多个关键词雷同的退化情况
3     swap(_elem[lo], _elem[lo + rand() % (hi - lo + 1)]); //任选一个元素与首元素交换
4     T pivot = _elem[lo]; //以首元素为候选轴点——经以上交换，等效于随机选取
5     while (lo < hi) { //从向量的两端交替地向中间扫描
6         while (lo < hi)
7             if (pivot < _elem[hi]) //在大于pivot的前提下
8                 hi--; //向左拓展右端子向量
9             else //直至遇到不大于pivot者
10                { _elem[lo++] = _elem[hi]; break; } //将其归入左端子向量
11        while (lo < hi)
12            if (_elem[lo] < pivot) //在小于pivot的前提下
13                lo++; //向右拓展左端子向量
14            else //直至遇到不小于pivot者
15                { _elem[hi--] = _elem[lo]; break; } //将其归入右端子向量
16    } //assert: lo == hi
17    _elem[lo] = pivot; //将备份的轴点记录置于前、后子向量之间
18    return lo; //返回轴点的秩
19 }
```

代码12.3 轴点构造算法（版本B）

较之版本A，版本B主要是调整了两个内循环的终止条件。以前一内循环为例，原条件

```
pivot <= _elem[hi]
```

在此更改为：

```
pivot < _elem[hi]
```

也就是说，一旦遇到重复元素，右端子向量随即终止拓展，并将右端重复元素转移至左端。因此，若将版本A的策略归纳为“勤于拓展、懒于交换”，版本B的策略则是“懒于拓展、勤于交换”。

■ 效果及性能

对照代码12.3不难验证，对于由重复元素构成的输入向量，以上版本B将交替地将右（左）侧元素转移至左（右）侧，并最终恰好将轴点置于正中央的位置。这就意味着，退化的输入向量能够始终被均衡的切分，如此反而转为最好情况，排序所需时间为 $O(n\log n)$ 。

当然，以上改进并非没有代价。比如，单趟partition()算法需做更多的元素交换操作。好在这并不影响该算法的线性复杂度。另外，版本B倾向于反复交换重复的元素，故它们在原输入向量中的相对次序更难保持，快速排序算法稳定性的不足更是雪上加霜。

§ 12.2 *选取与中位数

12.2.1 概述

■ k-选取

考查如下问题：

在任意一组可比较大小的元素中，如何找出由小到大次序为 k 者？

如图12.7(a)所示，也就是要从与这组元素对应的有序序列 S 中，找出秩为 k 的元素 $S[k]$ ，故称作选取（selection）问题。若将目标元素的秩记作 k ，则亦称作 k -选取（ k -selection）问题。以无序向量 $A = \{ 3, 13, 2, 5, 8 \}$ 为例，对应的有序向量为 $S = \{ 2, 3, 5, 8, 13 \}$ ，其中的元素依次与 $k = \{ 0, 1, 2, 3, 4 \}$ 相对应。



图12.7 选取与中位数

作为 k -选取问题的特例， 0 -选取即通常的最小值问题，而 $(n - 1)$ -选取问题即通常的最大值问题。这两个问题都有平凡的最优解，例如`List::selectMax()`（82页代码3.21）。

在允许元素重复的场合，秩为 k 的元素可能同时存在多个副本。此时不妨约定，其中任何一个都可作为解答输出。

■ 中位数

如图12.7(b)所示，在长度为 n 的有序序列 S 中，位序居中的元素 $S[\lfloor n/2 \rfloor]$ 称作中值或中位数（median）。例如，有序序列 $S = \{ 2, 3, \boxed{5}, 8, 13 \}$ 的中位数，为 $S[\lfloor 5/2 \rfloor] = S[2] = 5$ ；而有序序列 $S = \{ 2, 3, 5, \boxed{8}, 13, 21 \}$ 的中位数，则为 $S[\lfloor 6/2 \rfloor] = S[3] = 8$ 。

即便对于尚未排序的序列，也可定义中位数——也就是在对原数据集排序之后，对应的有序序列的中位数。例如，无序序列 $A = \{ 3, 13, 2, \boxed{5}, 8 \}$ 的中位数为元素 $A[3] = 5$ 。

由于中位数可将原数据集（原问题）划分为大小明确、规模相仿且彼此独立的两个子集（子问题），故能否高效地确定中位数，将直接关系到采用分治策略的算法能否高效地实现。

■ 蛮力算法

由中位数的定义，可直接得到查找中位数的如下直觉算法：对所有元素做排序，将其转换为有序序列 S ；于是， $S[\lfloor n/2 \rfloor]$ 便是所要找的中位数。然而根据2.7.5节的结论，该算法在最坏情况下需要 $\Omega(n \log n)$ 时间。于是，基于该算法的任何分治算法，时间复杂度都会不低于：

$$T(n) = n \log n + 2 \cdot T(n/2) = \Theta(n \log^2 n)$$

这一效率难以令人接受。

综上所述，中位数查找问题的挑战恰恰就在于：

如何在避免全排序的前提下，在 $\Theta(n \log n)$ 时间内找出中位数？

不难看出，所谓中位数查找问题，也可以理解为是选取问题在 $k = \lfloor n/2 \rfloor$ 时的特例。稍后我们将看到，中位数查找问题既是选取问题的特例，同时也是选取问题中的难度最大者。

以下先结合若干特定情况讨论中位数的定位算法，然后再回到一般性的选取问题。